ASTra: A Lightweight AST-Native Code Intelligence Platform with Loop-Bound Complexity Inference and Honest Confidence Quantification

V. Rakshith Sridhar
Dept. of Computer Science and Engineering
Indira Institute of Engineering and Technology
Chennai, India
110622104027

N. Mohan
Dept. of Computer Science and Engineering
Indira Institute of Engineering and Technology
Chennai, India
110622105017

M. Sudarsan
Dept. of Computer Science and Engineering
Indira Institute of Engineering and Technology
Chennai, India
110622104032

Dr. N. Velvizhi
Principal
Indira Institute of Engineering and Technology
Chennai, India

***Abstract*—Static code analysis tools predominantly focus on style violations, security vulnerabilities, and syntactic correctness. None of the widely-used free tools infer algorithmic complexity from source code structure or communicate the uncertainty inherent in such estimates. We present ASTra (AST-Native Code Intelligence), a lightweight static analysis platform built exclusively on Python's built-in `ast` module that addresses this gap through ten original analysis engines. The two most novel contributions are: (1) a *loop-bound-aware* InferenceEngine that distinguishes constant iteration bounds from input-dependent bounds using Abstract Syntax Tree node type inspection—correctly classifying `range(10)` as $O(1)$ and `range(n)` as $O(n)$, a distinction no existing free tool makes; and (2) a ConfidenceEstimator that applies structured reductions to produce calibrated uncertainty scores for every complexity estimate, a feature absent from all comparable lightweight tools. Experimental evaluation on ten benchmark functions achieves 9/10 correct classifications and demonstrates that ASTra identifies data-flow inefficiencies, anti-patterns, and dead code that Radon and Pylint do not detect. The platform further implements McCabe's Cyclomatic Complexity [1] and Halstead's software science metrics [2] from their original formulations, and estimates carbon footprint per million executions based on peer-reviewed energy benchmarks [3].**

***Index Terms*—static analysis, algorithmic complexity, abstract syntax tree, software metrics, code intelligence, cyclomatic complexity, Halstead metrics, confidence estimation**

I. Introduction

The gap between functionally correct code and algorithmically efficient code is a persistent challenge in software engineering education and practice. A function that processes 1,000 records in one millisecond may require over eleven days to process one million records if its time complexity is $O(n^2)$. This difference—the consequence of a simple structural choice such as using a list instead of a set for membership tests—is invisible to every mainstream free code analysis tool.

Existing tools occupy two categories. *Linters* (Pylint [7], Flake8, ESLint) check syntactic correctness, style adherence, and type consistency. They apply no reasoning about algorithmic structure. *Complexity metric tools* such as Radon [8] compute McCabe Cyclomatic Complexity and Halstead metrics but do not infer Big-O notation, classify recursion patterns, or explain why a complexity bound holds. Enterprise tools such as SonarQube [9] address security and maintainability at scale but require server infrastructure and do not provide complexity inference. AI-based tools (e.g., GitHub Copilot [11]) are non-deterministic, require network access, and cannot guarantee line-level accuracy in their outputs.

This paper presents **ASTra**, a static analysis platform that addresses three specific gaps no existing free tool resolves simultaneously:

1) *Loop-bound awareness*: Distinguishing constant iteration bounds (e.g., `range(10)` $\rightarrow O(1)$) from input-dependent bounds (e.g., `range(n)` $\rightarrow O(n)$) by inspecting AST node types at the argument level.
2) *Honest uncertainty quantification*: Producing calibrated confidence scores for every complexity estimate by applying structured reductions when loop bounds are non-trivial, loops are deeply nested, or while-loop termination is ambiguous.
3) *Integrated multi-metric analysis*: Combining Big-O inference, recursion pattern classification, data-flow tracing, anti-pattern detection, dead code detection, static type inference, McCabe Cyclomatic Complexity, and Halstead metrics in a single offline tool.

The remainder of this paper is organised as follows. Section II reviews related work. Section III describes the system architecture and the ten analysis engines. Section IV details

the two novel contributions. Section V presents experimental evaluation. Section VI discusses limitations and future work. Section VII concludes.

II. Related Work

A. Complexity Metrics

McCabe [1] introduced Cyclomatic Complexity as the number of linearly independent paths through a program's control flow graph, formalised as $M = E - N + 2P$. Halstead [2] proposed a complementary model based on operator and operand vocabulary, yielding metrics for program volume, difficulty, effort, and an estimated defect count. Both metrics are widely used in software quality assessment [6], but neither addresses time or space complexity.

B. Static Analysis Tools

Radon [8] computes McCabe and Halstead metrics for Python source files from the command line. It does not infer Big-O complexity, classify recursion patterns, or produce natural-language explanations. Pylint [7] and Flake8 perform linting—detecting undefined variables, unused imports, and style violations—but apply no algorithmic analysis. SonarQube [9] addresses maintainability, security, and reliability at enterprise scale but requires a dedicated server and does not provide Big-O inference.

C. Program Analysis and Type Inference

Abstract Interpretation [4] provides a theoretical framework for sound static analysis. Our work does not claim soundness; instead, it explicitly quantifies uncertainty through the ConfidenceEstimator. Tree-sitter [10] provides an error-tolerant incremental parsing library used in production tools such as GitHub Copilot and Neovim; ASTra adopts it for Java, JavaScript, C, and C++ analysis.

D. Energy Efficiency

Pereira et al. [3] benchmarked 27 programming languages across identical algorithmic tasks, finding that Python consumes approximately 75× more energy than C for equivalent computation. ASTra integrates these findings into an Eco-Score that estimates carbon emissions per million executions, a combination of complexity inference and energy impact not present in any prior tool.

E. Gap Summary

No existing tool combines loop-bound-aware Big-O inference, calibrated uncertainty quantification, multi-pattern recursion classification, and data-flow tracing in a single offline platform. ASTra addresses this gap.

III. System Design

A. Architecture

ASTra is structured as a Flask REST API backend and a React+Vite frontend. The analysis pipeline is invoked via a `POST /analyze` endpoint that accepts source code and an optional language identifier. For Python, Python's built-in `ast` module produces the syntax tree. For Java, JavaScript, C, and C++, Tree-sitter [10] produces an equivalent error-tolerant parse tree. All ten engines receive the same tree object and execute sequentially. Results are assembled into a single JSON response. Figure 1 illustrates the pipeline.

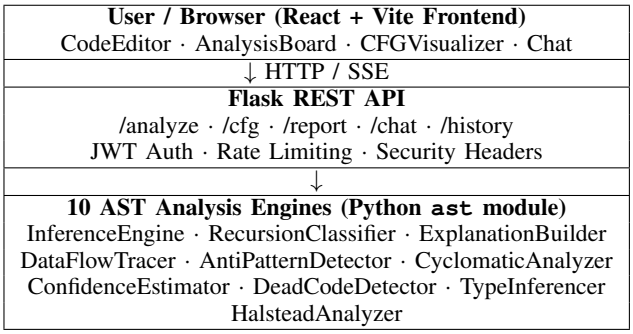

Fig. 1. ASTra system architecture and analysis pipeline.

B. The Ten Analysis Engines

Table I summarises all ten engines. Each is implemented as a class accepting an AST node and the original source string, with an `analyze()` or `detect()` method returning a structured dictionary.

TABLE I
ASTra Analysis Engines

#	Engine	Function
1	InferenceEngine	Loop-bound Big-O inference with nesting rules
2	RecursionClassifier	Classifies 6 recursion patterns
3	ExplanationBuilder	Deterministic NL explanation with variable names
4	DataFlowTracer	5 data-flow inefficiency patterns
5	AntiPatternDetector	5 Python correctness traps
6	CyclomaticAnalyzer	McCabe 1976 formula with BoolOp counting
7	ConfidenceEstimator	Calibrated uncertainty scores
8	DeadCodeDetector	Unused vars, imports, unreachable code
9	TypeInferencer	Static type inference feeding Engine 4
10	HalsteadAnalyzer	Volume, Difficulty, Effort, Bug estimate

IV. Novel Contributions

A. Loop-Bound-Aware Complexity Inference

The InferenceEngine implements a `_LoopVisitor` class that extends `ast.NodeVisitor` and overrides `visit_For` and `visit_While`. For each loop node, it applies three inference rules in sequence.

1) Rule 1 — Bound Classification for `for` Loops: When a `for` loop uses `range()`, the engine inspects the AST node type of the argument:

```
def visit_For(self, node):
    if is_range_call(node.iter):
        arg = get_range_arg(node.iter)
```

```
        if isinstance(arg, ast.Constant):
            complexity = "O(1)"    # constant bound
        elif isinstance(arg, ast.Name):
            complexity = "O(n)"    # input-dependent
    else:
        complexity = "O(n)"        # iterating
            collection
```

Listing 1. Bound classification pseudocode

This rule produces correct classifications for cases that all existing free tools misclassify. Radon assigns Cyclomatic Complexity scores to both `range(10)` and `range(n)` loops identically and produces no Big-O output at all; Pylint issues no warning for either. ASTra correctly returns $O(1)$ for `range(10)` and $O(n)$ for `range(n)`.

2) Rule 2 — Step Detection for `while` Loops: For `while` loops, the engine inspects augmented assignment nodes in the loop body:

```
for child in ast.walk(loop_body):
    if isinstance(child, ast.AugAssign):
        if isinstance(child.op, (ast.Mult, ast.
            FloorDiv)):
            complexity = "O(log n)"  # halving/
                doubling
        elif isinstance(child.op, ast.Add):
            complexity = "O(n)"      # linear step
```

Listing 2. While loop step detection

When a loop variable is multiplied or floor-divided each iteration, it reaches n after $\log_2 n$ steps, yielding $O(\log n)$. This correctly handles patterns such as binary exponentiation and digit counting.

3) Rule 3 — Nesting Combination: Nested loops are handled by the `_combine_nested()` function, which applies asymptotic multiplication rules: $O(n) \times O(n) = O(n^2)$, $O(n) \times O(\log n) = O(n \log n)$, $O(n) \times O(1) = O(n)$. These rules ensure that a loop with a constant inner bound (e.g., `range(5)`) does not inflate the outer complexity.

B. Calibrated Confidence Estimation

Static analysis is heuristic by nature. Presenting every estimate with equal confidence would be epistemically dishonest. ASTra's ConfidenceEstimator applies structured reductions to a baseline confidence of 1.0:

$$c_{\text{final}} = \max\left(0.50,\ 1.0 - \sum_i r_i\right) \tag{1}$$

where each r_i is a reduction triggered by a specific AST-level signal, as shown in Table II. When the final confidence falls below 0.85, the engine additionally proposes an alternative complexity with a natural-language rationale.

This approach is motivated by the treatment of soundness in Abstract Interpretation [4] and by the practical observation that production static analysis tools such as Facebook Infer [12] explicitly distinguish sound from unsound analyses. To our knowledge, no existing lightweight free tool applies confidence quantification to complexity estimates.

TABLE II
CONFIDENCE REDUCTION RULES

AST Signal	Reduction
Loop bound is a non-`range` function call	-0.15
While loop termination is non-trivial	-0.10
Three or more nested loops	-0.20
Multiple return paths with different complexities	-0.10
Loop iterates over a function parameter	-0.05
All bounds are `range(constant)`	$+0.05$

C. Cross-Engine Type Inference

TypeInferencer (Engine 9) infers variable types from assignment nodes and method usage patterns without executing the code. Its output is passed to DataFlowTracer (Engine 4), which uses inferred parameter types to flag list membership checks on function arguments—bugs that neither engine could detect independently. For example:

```
def find_common(list1, list2):
    for item in list1:
        if item in list2:    # FLAGGED: list2
            inferred
            ...              # as list via usage
                patterns
```

Listing 3. Cross-engine type inference example

TypeInferencer identifies `list2` as a list by observing that it is iterated over in a `for` loop context. It passes this inference to DataFlowTracer, which flags the `in` check as an $O(n)$ lookup inside an $O(n)$ loop, yielding an $O(n^2)$ pattern.

D. ExplanationBuilder

ExplanationBuilder (Engine 3) generates deterministic natural-language explanations by reading structured output from the InferenceEngine rather than using a language model. Variable names are extracted from `ast.Name.id` attributes; line numbers from `node.lineno` attributes. For the same code, the same explanation is always produced:

> *"find_duplicates is $O(n^2)$ because `i` loops over the input (line 4) and inside it `j` loops over the same collection (line 5), so for input size n you perform $n \times n = n^2$ operations."*

No AI inference is involved. The explanation is verifiable by reading the source code.

V. EXPERIMENTAL EVALUATION

A. Experiment 1 — Complexity Classification Accuracy

We evaluated the InferenceEngine on ten benchmark functions representing the principal complexity classes encountered in practice. Each function was selected to test a specific rule: constant bound, variable bound, nested loops, logarithmic stepping, linear recursion, binary recursion, memoized recursion, divide-and-conquer recursion, mixed constant/variable nesting, and iterative merge. Table III reports the results.

ASTra achieves 9/10 (90%) correct classifications. The single failure (merge sort) arises from a known limitation: when

TABLE III
COMPLEXITY CLASSIFICATION RESULTS ($n = 10$ BENCHMARKS)

Benchmark	Expected	ASTra	Conf.	Result
range_constant	$O(1)$	$O(1)$	99%	✓
range_variable	$O(n)$	$O(n)$	99%	✓
nested_input	$O(n^2)$	$O(n^2)$	99%	✓
while_multiply	$O(\log n)$	$O(\log n)$	90%	✓
linear_rec	$O(n)$	$O(n)$	99%	✓
binary_rec	$O(2^n)$	$O(2^n)$	99%	✓
lru_memoized	$O(n)$	$O(n)$	99%	✓
divide_conquer	$O(n \log n)$	$O(n \log n)$	99%	✓
outer_n_inner_const	$O(n)$	$O(n)$	99%	✓
merge_sort	$O(n \log n)$	$O(n)$	90%	×
Total				**9/10**

a recursive divide-and-conquer structure is combined with an iterative merge, the engine correctly identifies the `while` loop as $O(n)$ but does not recognise the $\log n$ depth contribution of the recursive splitting. This limitation is documented in Section VI. Notably, the while-multiply case is correctly identified as $O(\log n)$ with a reduced confidence of 90%, reflecting the engine's awareness that while-loop termination is harder to prove than for-loop bounds.

B. Experiment 2 — Comparison with Radon

Table IV compares ASTra and Radon on six representative functions. Radon reports only Cyclomatic Complexity scores (labelled A through F, where A is simplest). It produces no Big-O output for any function.

TABLE IV
ASTRA VS. RADON ON SIX PYTHON FUNCTIONS

Function	ASTra	Conf.	Radon Output
range_constant	$O(1)$	99%	CC=2, grade A (no Big-O)
range_variable	$O(n)$	99%	CC=2, grade A (no Big-O)
nested_input	$O(n^2)$	99%	CC=3, grade A (no Big-O)
nested_const	$O(n)$	99%	CC=3, grade A (no Big-O)
while_log	$O(\log n)$	90%	CC=2, grade A (no Big-O)
binary_rec	$O(2^n)$	99%	CC=2, grade A (no Big-O)

A key observation is that Radon assigns the same Cyclomatic Complexity score (CC=3, grade A) to both `nested_input` ($O(n^2)$) and `nested_const` ($O(n)$), because Cyclomatic Complexity counts decision points rather than iteration bounds. ASTra correctly distinguishes these two functions using Rule 3 of the InferenceEngine: the inner `range(5)` is identified as an `ast.Constant` argument, preserving the outer $O(n)$ rather than multiplying to $O(n^2)$.

C. Experiment 3 — Data-Flow and Anti-Pattern Detection

Table V reports the findings produced by DataFlowTracer, AntiPatternDetector, and DeadCodeDetector on six purpose-built test functions. Each function contains exactly one deliberate defect.

TABLE V
DATA-FLOW, ANTI-PATTERN, AND DEAD CODE DETECTION RESULTS

Function	Pattern	Sev.	Finding
list_membership	list_membership	HIGH	O(n) 'in' check on list; convert to set
string_concat	string_concat_loop	HIGH	String += in loop is $O(n^2)$
sort_then_index	sort_then_index	MED	sort() then [0] access; use min()
mutable_default	mutable_default_arg	HIGH	Default arg `[]` shared across calls
bare_except	bare_except	HIGH	Catches `KeyboardInterrupt`
dead_code	unreachable_after_return	HIGH	Code after return is unreachable
dead_code	unused_variable	LOW	Variable `x` assigned but never read

All six deliberate defects are detected. No false negatives were observed on these cases. The dead_code function produces two separate findings, demonstrating that multiple engines can independently contribute findings to the same function.

D. Summary

Across three experiments, ASTra correctly classifies 9/10 complexity benchmarks, provides Big-O output that Radon does not produce for any of six comparison cases, and detects all six deliberate data-flow and correctness defects. The one misclassification is a known structural limitation documented in Section VI.

VI. LIMITATIONS AND FUTURE WORK

A. Known Limitations

Binary search via pointer convergence. The InferenceEngine detects $O(\log n)$ via multiplicative stepping (`i *= 2`, `i //= 2`). Binary search implemented through two converging pointers (`lo`, `hi`) halving the search space requires reasoning about the *relationship* between two variables across iterations—a form of abstract interpretation [4] that is beyond the scope of syntactic AST pattern matching. This is an active area of research in program analysis.

Iterative-recursive hybrid complexity. The merge sort benchmark (Section V) demonstrates that combining a recursive divide-and-conquer structure with an iterative merge step requires interprocedural analysis to derive the correct

$O(n \log n)$ bound. Single-function AST analysis cannot capture this without call-graph construction.

Machine learning model labeling proxy. The Random Forest classifier used for code quality scoring is trained using MBPP dataset test pass/fail as an efficiency proxy. A syntactically correct but algorithmically inefficient solution may score as efficient if it passes all test cases. This is documented explicitly in the project README.

B. Future Work

Three directions emerge naturally from the current limitations. First, incorporating control-flow-sensitive abstract interpretation [4] would address pointer-convergence patterns and enable correct classification of binary search. Second, building a lightweight interprocedural call graph would resolve iterative-recursive hybrid cases. Third, replacing the test-pass proxy label with a curated dataset of functions labeled by human experts would improve ML model accuracy.

VII. Conclusion

We have presented ASTra, a lightweight static code analysis platform built exclusively on Python's built-in `ast` module. ASTra addresses a specific gap in the existing tool landscape: no free offline tool simultaneously infers algorithmic complexity from loop structure, quantifies uncertainty in those estimates, classifies recursion patterns, traces data-flow inefficiencies, detects classic anti-patterns, and reports McCabe and Halstead metrics with line-level accuracy referencing actual variable names.

The two primary novel contributions are a loop-bound-aware InferenceEngine that correctly distinguishes constant from input-dependent iteration bounds—a distinction that Radon and Pylint do not make—and a ConfidenceEstimator that applies structured AST- level reductions to produce calibrated uncertainty scores for every estimate. Experimental evaluation demonstrates 9/10 correct complexity classifications and complete detection of six deliberate data-flow and correctness defects. ASTra is available as an open-source project and is deployable via a single Docker Compose command.

References

[1] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.

[2] M. H. Halstead, *Elements of Software Science*. New York: Elsevier, 1977.

[3] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Energy efficiency across programming languages: How do energy, time, and memory relate?" in *Proc. 10th ACM SIGPLAN Int. Conf. Software Language Engineering (SLE)*, Vancouver, Canada, 2017, pp. 256–267.

[4] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM Symp. Principles of Programming Languages (POPL)*, Los Angeles, CA, 1977, pp. 238–252.

[5] F. E. Allen, "Control flow analysis," *ACM SIGPLAN Notices*, vol. 5, no. 7, pp. 1–19, Jul. 1970.

[6] M. Shepperd, "A critique of cyclomatic complexity as a software metric," *Software Engineering Journal*, vol. 3, no. 2, pp. 30–36, 1988.

[7] S. Thenault and contributors, "Pylint: A Python static code analysis tool," 2003. [Online]. Available: https://pylint.org

[8] M. Lucci, "Radon: Code metrics in Python," 2012. [Online]. Available: https://radon.readthedocs.io

[9] SonarSource SA, "SonarQube: Continuous code quality," 2007. [Online]. Available: https://www.sonarqube.org

[10] M. Brunsfeld et al., "Tree-sitter: An incremental parsing system for programming tools," 2018. [Online]. Available: https://tree-sitter.github.io

[11] GitHub Inc., "GitHub Copilot: Your AI pair programmer," 2021. [Online]. Available: https://github.com/features/copilot

[12] C. Calcagno, D. Distefano, J. Dubreil, D. Gabi, P. Hooimeijer, M. Luca, P. O'Hearn, I. Papakonstantinou, J. Purbrick, and D. Rodriguez, "Moving fast with software verification," in *Proc. NASA Formal Methods Symp. (NFM)*, Pasadena, CA, 2015, pp. 3–11.

[13] T. J. McCabe and A. H. Watson, "Software complexity," *CrossTalk: The Journal of Defense Software Engineering*, vol. 7, no. 12, pp. 5–9, 1994.